



ACTIVIDAD 1: INVESTIGACION – INTRODUCCION A LAS BASES DE DATOS

Materia: Bases de datos I

Docente: Hugo Arnaldo Guzman

Estudiantes:

Ricardo Salvatierra Cardozo

Douglas Hernández Flor

Luis Rene Khonangh Sequeiro

Nicolas Soria Yabeta

Contenido

1. DEFINICIÓN DE BASE DE DATOS.....	2
2. CONCEPTOS Y TERMINOLOGÍAS.....	2
2.1.- TABLA.....	3
2.2.-PARTES DE UNA TABLA EN BASE DE DATOS:.....	3
2.3.-ESTRUCTURA DE LAS TABLAS.....	3
• ÍNDICE.....	4
2.4.- TIPOS DE TABLAS EN BASE DE DATOS.....	4
3. BASE DE DATOS RELACIONALES Y ANTERIORES MODELOS.....	5
3.1.- MODELOS ANTERIORES:.....	5
3.1.1.- <i>Modelo Jerárquico</i> :.....	5
3.1.2.- <i>Modelo de Red</i> :.....	5
3.2.- MODELO RACIONAL.....	5
3.2.1.- <i>Reglas de integridad del modelo relacional de base de datos</i>	5
3.2.2. <i>¿Cómo hacer un modelo relacional de base de datos?</i>	6
4. SEPARACIÓN DE LOS PROGRAMAS Y DATOS.....	9
4.1 SEPARACIÓN DE DATOS:.....	10
4.2.- SEPARACIÓN DE PROGRAMAS:.....	10
5. TRANSACCIÓN. PROPIEDADES DE LAS TRANSACCIONES.....	11
6. USUARIOS DE UNA BASE DE DATOS.....	12
6.1.- ADMINISTRADORES DE BASES DE DATOS (DBA): RESPONSABLES DE LA GESTIÓN Y EL MANTENIMIENTO DE LA BASE DE DATOS.....	12
6.2.- DESARROLLADORES:.....	12
6.3.- USUARIOS FINALES:.....	12
7. MAPA MENTAL.....	13
8. BIBLIOGRAFÍA.....	13

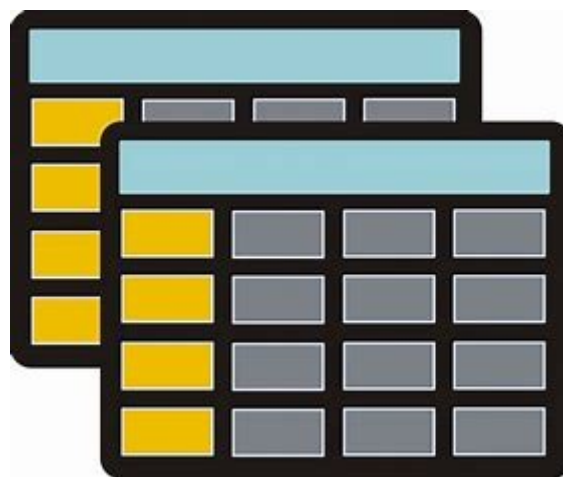
1. DEFINICIÓN DE BASE DE DATOS

Una base de datos es un conjunto organizado de datos que se almacenan y se gestionan electrónicamente. Las bases de datos permiten el almacenamiento, la recuperación y la manipulación eficiente de grandes cantidades de información. Se utilizan en diversas aplicaciones, desde sistemas de gestión empresarial hasta aplicaciones web y móviles. Se podría decir que una base de datos es una colección organizada de datos almacenados en un sistema informático, diseñada para facilitar su acceso, gestión y manipulación. Estos datos pueden ser de diferentes tipos, como números, caracteres, imágenes, audio, video, etc. Una base de datos proporciona una forma estructurada de almacenar y recuperar información, lo que permite a los usuarios acceder a los datos de manera eficientes y segura

Ejemplo para una empresa de ventas: Una base de datos de clientes de una empresa de ventas podría contener información como el nombre, la dirección, el número de teléfono, el correo electrónico, las compras anteriores, el historial de pedidos y otras características relevantes para el negocio.

2. CONCEPTOS Y TERMINOLOGÍAS

Cuando estudiamos el tema Base de Datos nos encontramos el término “tablas”, en primer lugar, debemos tener claro que una Base de Datos es una especie de almacén, en el cual podemos organizar y guardar gran cantidad de información para su posterior uso. Las Bases de Datos están compuestas por una o más tablas, estas a su vez están compuestas por campos y registros.



2.1.- TABLA

Una tabla en una base de datos es un conjunto de datos organizados en filas y columnas. Cada columna representa un campo específico, como un nombre o una dirección, y cada fila contiene un conjunto de datos para ese campo, como el nombre y la dirección de un cliente. Las tablas en una base de datos pueden estar relacionadas entre sí a través de campos comunes, lo que permite que se

vinculen automáticamente los datos de diferentes tablas cuando se realizan consultas o se crean informes.

Las tablas son uno de los componentes fundamentales de una base de datos y permiten almacenar grandes cantidades de información de manera organizada y eficiente.

2.2.-PARTES DE UNA TABLA EN BASE DE DATOS:

Para comprender mejor una tabla en una base de datos, es esencial conocer sus partes fundamentales. Aquí hay una breve descripción de estas partes:

- **Filas:** Cada fila en una tabla representa un registro o entrada de datos único. Por ejemplo, en una tabla de productos, cada fila podría representar un producto diferente con su nombre, precio y descripción.
- **Columnas:** Las columnas son campos que definen los tipos de datos que se almacenan en la tabla. Cada columna tiene un nombre que indica qué tipo de información contiene, como texto, número, fecha, etc.
- **Clave Primaria:** La clave primaria es una columna especial que contiene valores únicos para cada fila en la tabla. Ayuda a identificar y acceder rápidamente a registros específicos.
- **Claves Externas:** (Clave foránea) En algunas tablas, se utilizan claves externas para establecer relaciones con otras tablas en la base de datos. Esto permite la creación de vínculos entre datos relacionados.

2.3.-ESTRUCTURA DE LAS TABLAS

Las tablas están compuestas por campos y registros, en donde

- **Campo:** Se refiere al nombre de la columna. Es un dato único y además se le establece un tipo de dato.
- **Registro:** Se refiere a cada fila que conforma la tabla, dicho de otra manera, son los datos y registros que almacenamos. Cabe aclarar que en ocasiones pueden quedar datos nulos.
- **Índice:** Estructura que mejora la velocidad de las operaciones de búsqueda en una base de datos.



2.4.- TIPOS DE TABLAS EN BASE DE DATOS

Las tablas de datos son las más utilizadas y versátiles en una base de datos. Almacenan información principal, como registros de clientes, productos, transacciones financieras, etc. Estas tablas suelen tener una clave primaria que identifica de forma única cada registro.

- **Tablas de Configuración:**

Las tablas de configuración almacenan datos estáticos que se utilizan para personalizar y configurar la base de datos. Ejemplos de datos en estas tablas incluyen listas de estados, categorías de productos o tipos de documentos. Son cruciales para mantener la consistencia y uniformidad de la información.

- **Tablas de Relación:**

Estas tablas se utilizan para establecer relaciones entre otras tablas en la base de datos. Permiten vincular datos relacionados en diferentes tablas y, por lo tanto, realizar consultas complejas. Las claves externas son comunes en este tipo de tablas.

- **Tablas de Auditoría:**

Las tablas de auditoría registran cambios realizados en la base de datos, como quién modificó un registro, cuándo se realizó la modificación y qué cambios se hicieron. Son esenciales para el seguimiento y la seguridad de los datos.

- **Tablas Temporales:**

Las tablas temporales almacenan datos temporales o intermedios que se utilizan durante una sesión o proceso específico. Se borran automáticamente cuando ya no son necesarias.

3. BASE DE DATOS RELACIONALES Y ANTERIORES MODELOS

3.1.- Modelos anteriores:

Incluyen bases de datos jerárquicas y de red. Las bases de datos jerárquicas organizan los datos en una estructura de árbol, mientras que las bases de datos de red permiten relaciones más complejas mediante grafos.

3.1.1.- Modelo Jerárquico:

- Estructura: Organiza los datos en una estructura de árbol, con un solo nodo raíz y varios niveles de nodos hijos.
- Ventajas: Buena para representar relaciones uno a muchos.
- Desventajas: Rigidez en la estructura, difícil de modificar y escalar.

3.1.2.- Modelo de Red:

- Estructura: Similar al modelo jerárquico, pero permite que un nodo hijo tenga múltiples padres, formando una red.

- Ventajas: Más flexible que el modelo jerárquico.
- Desventajas: Complejidad en la gestión y consulta de datos

3.2.- Modelo racional

El modelo relacional se define como la base de datos que permite agrupar sus elementos de datos en una o más tablas independientes, que pueden relacionarse entre sí utilizando campos comunes a cada tabla relacionada.

El propósito del modelo relacional es brindar un método declarativo para especificar los datos y las consultas: los usuarios declaran directamente qué información contiene la base de datos y qué información quieren de ella.

Por otro lado, dejan que el software del sistema de gestión de bases de datos se encargue de describir las estructuras de datos para su almacenamiento y el procedimiento de recuperación para responder las consultas.

La mayoría de las bases de datos relacionales usan el lenguaje SQL (Structured Query Language): Lenguaje estándar para la gestión y consulta de bases de datos relacionales.

3.2.1.-Reglas de integridad del modelo relacional de base de datos

Al diseñar el modelo relacional, se definen algunas condiciones que deben cumplirse en la base de datos, denominadas reglas de integridad.

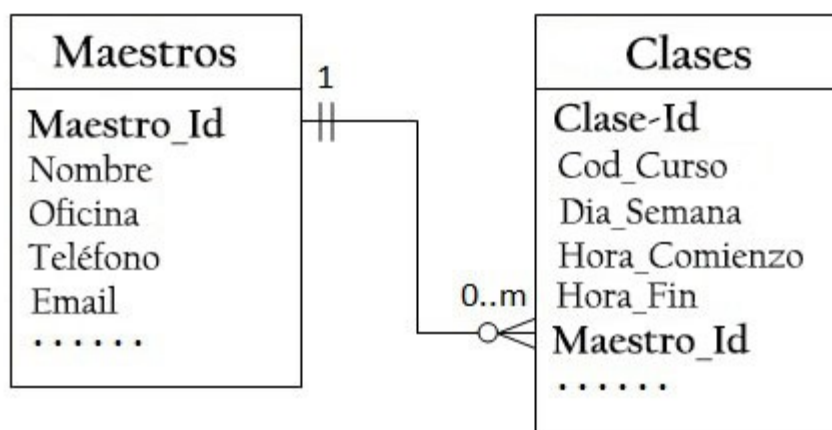
- **Integridad de la clave.** La clave primaria debe ser única para todas las tuplas y no puede tener el valor nulo (NULL). De lo contrario, no podrá identificar la fila de forma exclusiva. Para una clave compuesta por varias columnas, ninguna de esas columnas puede contener NULL.
- **Integridad referencial.** Cada valor de una clave externa debe coincidir con un valor de la clave primaria de la tabla referenciada o primaria. En la tabla secundaria solo se podrá insertar una fila con una clave externa si ese valor existe en una tabla primaria. Si el valor de la clave cambia en la tabla primaria, por actualizarse o eliminarse la fila, entonces todas las filas en las tablas secundarias con esta clave externa deben actualizarse o eliminarse.

3.2.2¿Cómo hacer un modelo relacional de base de datos?

- **Recopilar datos.** Se deben recopilar los datos necesarios para almacenarlos en la base de datos. Estos datos se dividen en diferentes tablas. Se debe elegir un tipo de datos apropiado para cada columna. Por ejemplo: números enteros, números de punto flotante, texto, fecha, etc.
- **Definir claves primarias.** Para cada tabla se debe elegir una columna (o pocas columnas) como clave primaria, que identificará de forma única cada fila de la tabla. La clave primaria también se utiliza para hacer referencia a otras tablas.
- **Crear relaciones entre tablas.** Una base de datos que consista en tablas independientes y no relacionadas tiene poco propósito. El aspecto más

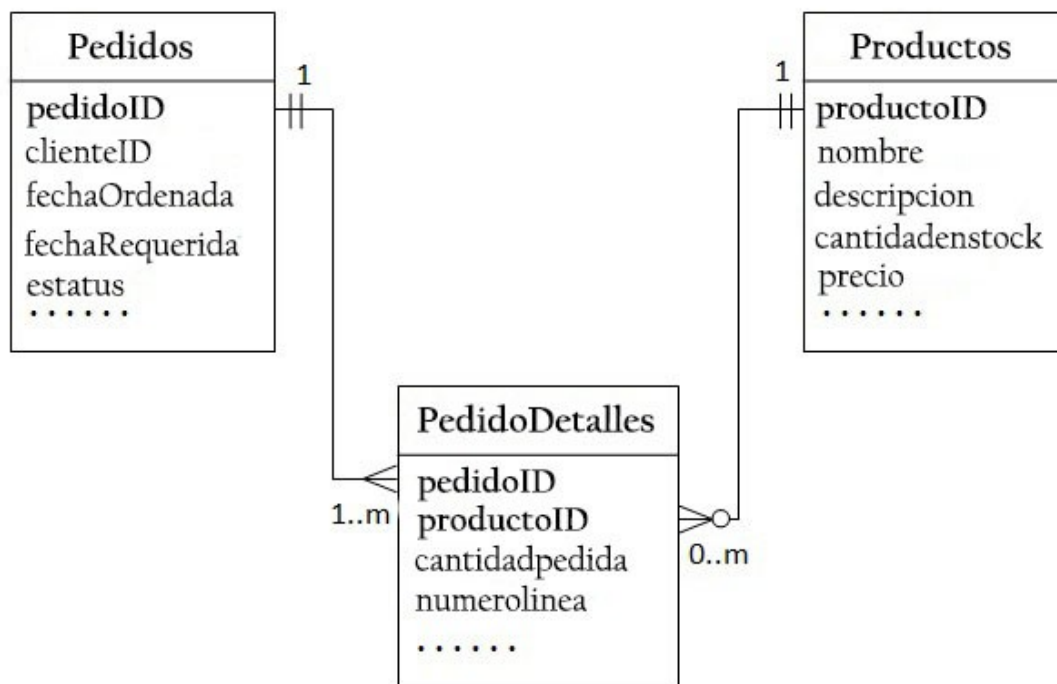
importante en el diseño de una base de datos relacional es identificar las relaciones entre las tablas. Los tipos de relación son:

- **Uno a muchos.** En una base de datos “Listado de clases”, un maestro puede enseñar en cero o más clases, mientras que una clase es impartida por un solo maestro. Este tipo de relación se conoce como uno a muchos. Esta relación no se puede representar en una sola tabla. En la base de datos “Listado de clases” se puede tener una tabla llamada Maestros, que almacena información sobre los maestros. Para almacenar las clases impartidas por cada maestro, se podrían crear columnas adicionales, pero se enfrentaría un problema: cuántas columnas crear. Por otro lado, si se tiene una tabla llamada Clases, que almacena información sobre una clase, se podrían crear columnas adicionales para almacenar la información sobre el maestro. Sin embargo, como un maestro puede enseñar en muchas clases, sus datos se duplicarían en muchas filas de la tabla Clases.
- **Diseñar dos tablas.** Por tanto, se necesitan diseñar dos tablas: una tabla Clases, para almacenar información sobre las clases, con Clase_Id como clave principal, y una tabla Maestros, para almacenar información sobre los maestros, con Maestro_Id como clave principal. Luego se puede crear la relación uno a muchos almacenando la clave primaria de la tabla Maestro (Maestro_Id) en la tabla Clases, como se ilustra a continuación.



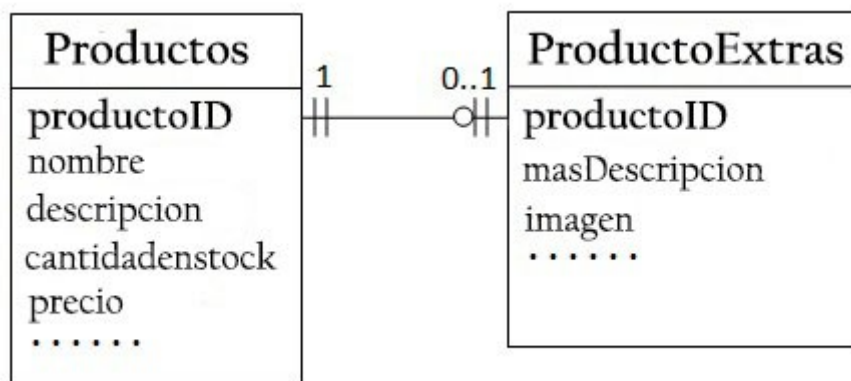
Diseñar c/tablas. La columna Maestro_Id en la tabla Clases es conocida como clave externa o clave secundaria. Para cada valor Maestro_Id en la tabla Maestro, puede haber cero o más filas en la tabla Clases. Para cada valor Clase_Id en la tabla Clases, hay solo una fila en la tabla Maestros.

- Muchos a muchos.** En una base de datos “Venta de productos”, el pedido de un cliente puede contener varios productos, y un producto puede aparecer en varios pedidos. Este tipo de relación se conoce como muchos a muchos. Se puede comenzar la base de datos “Venta de productos” con dos tablas: Productos y Pedidos. La tabla Productos contiene información sobre los productos, con productoID como clave primaria. Por otro lado, la tabla Pedidos contiene los pedidos del cliente, con pedidoID como clave primaria. No se pueden almacenar los productos pedidos dentro de la tabla Pedidos, ya que no se sabe cuántas columnas reservar para los productos. Tampoco se pueden almacenar los pedidos en la tabla Productos por la misma razón. Para admitir una relación muchos a muchos, se necesita crear una tercera tabla, conocida como tabla de unión (PedidoDetalles), donde cada fila representa un elemento de un pedido particular. Para la tabla PedidoDetalles, la clave primaria consta de dos columnas: pedidoID y productoID, identificando de forma única cada fila. Las columnas pedidoID y productoID en la tabla PedidoDetalles se utilizan para referenciar a las tablas Pedidos y Productos. Por tanto, también son claves externas en la tabla PedidoDetalles.



- Uno a uno.** En la base de datos “Venta de productos”, un producto puede tener información opcional, como descripción adicional y su imagen.

Mantenerla dentro de la tabla Productos generaría muchos espacios vacíos. Por tanto, se puede crear otra tabla (ProductoExtras) para almacenar los datos opcionales. Solo se creará un registro para los productos con datos opcionales. Las dos tablas, Productos y ProductoExtras, tienen una relación uno a uno. Para cada fila en la tabla Productos hay máximo una fila en la tabla ProductoExtras. Se debe usar el mismo productoID como clave principal para ambas tablas.



Actualmente existen muchos sistemas de manejo de bases de datos relacionales o RDBMS (Relational Data Base Management System) Sistemas de Gestión de Bases de Datos Relacionales, entre muchos tenemos

- Oracle
- Microsoft SQL Server
- MySQL
- PostgreSQL
-

3.2.3.- También tenemos las bases de datos no relacionales

Las bases de datos no relacionales, también conocidas como NoSQL (Not Only SQL), son una alternativa a las bases de datos relacionales y están diseñadas para manejar grandes volúmenes de datos no estructurados. En una empresa de marketing digital, los analistas de datos utilizan herramientas como SQL, Python y software de visualización de datos para analizar grandes volúmenes de datos de campañas publicitarias. Extraen información valiosa sobre el rendimiento de las campañas, identifican tendencias y generan informes que ayudan a los equipos de marketing a tomar decisiones informadas y optimizar sus estrategias.

y optimizar sus estrategias estructurados o semi-estructurados. Aquí te doy un resumen de los principales tipos de bases de datos NoSQL:

- CouchDB: También basada en documentos JSON, con un enfoque en la replicación y sincronización de datos.
- Redis: Conocida por su velocidad, es ideal para almacenamiento en caché y sesiones.
- DynamoDB: Ofrecida por Amazon Web Services, es altamente escalable y manejada.
- Apache Cassandra: Diseñada para manejar grandes volúmenes de datos distribuidos en múltiples servidores.
- HBase: Basada en Hadoop, es adecuada para aplicaciones que requieren acceso rápido a grandes cantidades de datos.
- Neo4j: Especializada en gestionar y consultar relaciones complejas entre datos.
- Amazon Neptune: Un servicio de base de datos de grafos completamente gestionado.

Otros Ejemplos

- Couchbase: Combina características de bases de datos de documentos y clave-valor.
- Elasticsearch: Utilizada principalmente para búsquedas y análisis de datos en tiempo real.

4. SEPARACIÓN DE LOS PROGRAMAS Y DATOS

La separación de programas y datos es un principio fundamental en el diseño de sistemas de información. Este principio asegura que los datos se gestionen de manera independiente de las aplicaciones que los utilizan. Esto facilita la gestión, la seguridad y la integridad de los datos, permitiendo que múltiples aplicaciones accedan a los mismos datos sin interferencias.

La separación de programas y datos es una buena práctica en la creación y mantenimiento de aplicaciones informáticas. Esta separación se logra mediante la implementación de arquitecturas y tecnologías que mantienen los datos y los programas en lugares diferentes, lo que permite una mejor organización, gestión y seguridad de la información.

A continuación, te presento algunos beneficios de la separación de programas y datos:

4.1 Separación de Datos:

- **Mejora la organización y gestión de la información:** Al mantener los datos separados, se puede tener una mejor visión de la estructura y el contenido de la información almacenada, lo que facilita la gestión y el análisis de los datos.
- **Aumenta la seguridad de los datos:** La separación de los datos hace que sean más difíciles de acceder o manipular de manera no autorizada, lo que mejora la seguridad de la información.
- **Facilita la escalabilidad:** Al mantener los datos separados, se puede agregar más capacidad o recursos sin afectar negativamente la aplicación o el sistema en el que se encuentra.
- **Aumenta la flexibilidad:** La separación de los datos permite la integración de diferentes sistemas y aplicaciones sin afectar la estructura de los datos o la aplicación principal.

4.2.- Separación de Programas:

- **Mejora el modularidad:** La separación de programas permite crear módulos o aplicaciones individuales que pueden ser desarrollados, mantenidos y actualizados de manera independiente.
- **Aumenta la escalabilidad:** Al mantener los programas separados, se puede agregar más capacidad o recursos sin afectar negativamente la aplicación o el sistema en el que se encuentra.
- **Facilita la prueba y depuración:** La separación de los programas hace que sea más fácil probar y depurar las aplicaciones individuales sin afectar otras partes del sistema.
- **Aumenta la seguridad:** La separación de los programas reduce el riesgo de vulnerabilidades o ataques que puedan afectar a múltiples aplicaciones o sistemas.

Ejemplo 1: Aplicación de Gestión de Inventarios

Contexto: Una empresa utiliza una aplicación para gestionar su inventario de productos.

Datos: La base de datos contiene tablas como Productos, Proveedores, Categorías, y Pedidos.

Programas: La aplicación tiene módulos para agregar, actualizar, y eliminar productos, así como para generar informes de inventario.

Separación: Los datos de los productos se almacenan en la base de datos y no están directamente accesibles por los usuarios finales.

Los programas de la aplicación interactúan con la base de datos a través de consultas SQL y procedimientos almacenados, asegurando que cualquier cambio en la estructura de la base de datos no afecte la lógica de la aplicación.

En resumen, la separación de programas y datos es una buena práctica que puede mejorar la organización, gestión y seguridad de la información, así como f

acilitar la escalabilidad, el modularidad y la prueba y depuración de las aplicaciones.

5. TRANSACCIÓN. PROPIEDADES DE LAS TRANSACCIONES

Una transacción es una unidad de trabajo que se ejecuta de manera completa o no se ejecuta en absoluto. Las propiedades de las transacciones se conocen como ACID:

5.1.-Atomicidad: La transacción se ejecuta completamente o no se ejecuta en absoluto.

- Ejemplo: Imagina una transacción bancaria que transfiere dinero de una cuenta a otra. La transacción incluye dos operaciones: debitar una cuenta y acreditar otra. La atomicidad asegura que ambas operaciones se completen exitosamente o ninguna de ellas se ejecute. Si ocurre un error después de debitar la primera cuenta pero antes de acreditar la segunda, la transacción se revierte y ninguna cuenta se ve afectada.

5.2.- Consistencia: La transacción lleva la base de datos de un estado válido a otro estado válido.

- Ejemplo: Supongamos que una base de datos tiene una restricción que asegura que el saldo de una cuenta no puede ser negativo. Una transacción que debita una cuenta debe verificar esta restricción antes de completar la operación. Si la transacción intenta debitar más dinero del que hay disponible, se cancela para mantener la consistencia de la base de datos.

5.3.- Aislamiento: Las transacciones concurrentes no interfieren entre sí.

- Ejemplo: Considera dos transacciones concurrentes en una tienda en línea: una que actualiza el inventario después de una venta y otra que consulta el inventario para mostrar la disponibilidad de productos. El aislamiento asegura que la transacción de consulta no vea un estado intermedio del inventario mientras la transacción de actualización está en progreso. Cada transacción se ejecuta como si fuera la única en el sistema.

5.4.- Durabilidad: Una vez que una transacción se ha completado, sus cambios son permanentes.

- ejemplo: Imagina una transacción que registra una nueva orden en un sistema de comercio electrónico. Una vez que la transacción se completa y se confirma, la orden debe permanecer registrada en la base de datos i

ncluso si ocurre una falla del sistema inmediatamente después. La durabilidad garantiza que los cambios realizados por la transacción sean permanentes y persistentes.

Estos ejemplos ilustran cómo las propiedades ACID aseguran la integridad y confiabilidad de las transacciones en un sistema de bases de datos.

6. USUARIOS DE UNA BASE DE DATOS

Los usuarios de una base de datos pueden clasificarse en varios tipos:

6.1.- Administradores de bases de datos (DBA): Responsables de la gestión y el mantenimiento de la base de datos.

En una empresa de comercio electrónico, los DBAs son responsables de asegurar que la base de datos que almacena la información de los productos, clientes y pedidos esté siempre disponible y funcionando correctamente. Realizan tareas como la configuración de copias de seguridad, la optimización del rendimiento de las consultas y la implementación de medidas de seguridad para proteger los datos sensibles.

6.2.- Desarrolladores: Crean aplicaciones que interactúan con la base de datos.

- Los desarrolladores en un startup de tecnología financiera crean aplicaciones móviles y web que permiten a los usuarios gestionar sus cuentas bancarias, realizar transferencias y consultar su historial de transacciones. Utilizan lenguajes de programación como Python o JavaScript para escribir el código que interactúa con la base de datos, asegurando que las operaciones se realicen de manera eficiente y segura.

6.3.- Usuarios finales: Utilizan las aplicaciones para realizar tareas específicas, como consultas y actualizaciones de datos.

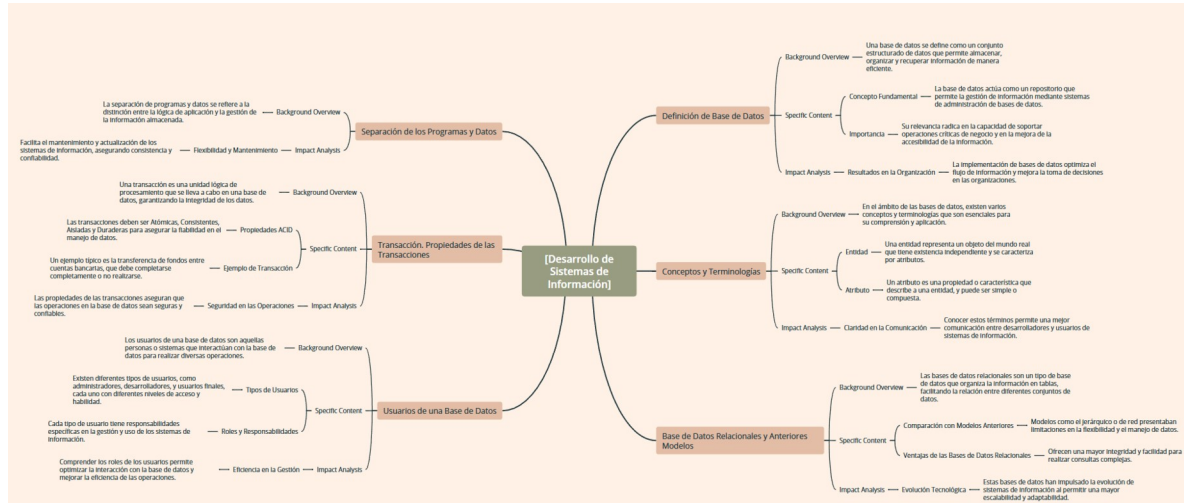
- En una universidad, los usuarios finales incluyen a los estudiantes y profesores que utilizan un sistema de gestión académica para inscribirse en cursos, consultar calificaciones y acceder a materiales de estudio. Estos usuarios interactúan con la base de datos a través de una interfaz de usuario amigable, sin necesidad de conocer los detalles técnicos de cómo se almacenan y gestionan los datos.

6.4.- Analistas de datos: Utilizan herramientas de análisis para extraer información y generar informes a partir de los datos almacenados.

- En una empresa de marketing digital, los analistas de datos utilizan herramientas como SQL, Python y software de visualización de datos para analizar grandes volúmenes de datos de campañas publicitarias. Extraen

información valiosa sobre el rendimiento de las campañas, identifican tendencias y generan informes que ayudan a los equipos de marketing a tomar decisiones informadas y optimizar sus estrategias.

7. MAPA MENTAL



8. BIBLIOGRAFÍA

http://intranet.utvm.edu.mx/biblioteca/biblioteca-virtual/utvm/tic/polilibros/Base%20de%20Datos%20I/Contenido/1_4_5_Usuarios_BD.htm

<https://www.datdata.com/blog/terminos-comunes-sql-bases-datos>

<https://www.oracle.com/mx/database/what-is-database/>

<https://www.oracle.com/ar/database/what-is-a-relational-database/>

<https://ayudaleyprotecciondatos.es/bases-de-datos/modelos/>

<https://basesdedatos.wordpress.com/arquitectura-de-una-base-de-datos/>

<https://appmaster.io/es/blog/que-es-una-transaccion-de-base-de-datos>

http://intranet.utvm.edu.mx/biblioteca/biblioteca-virtual/utvm/tic/polilibros/Base%20de%20Datos%20I/Contenido/1_4_5_Usuarios_BD.htm